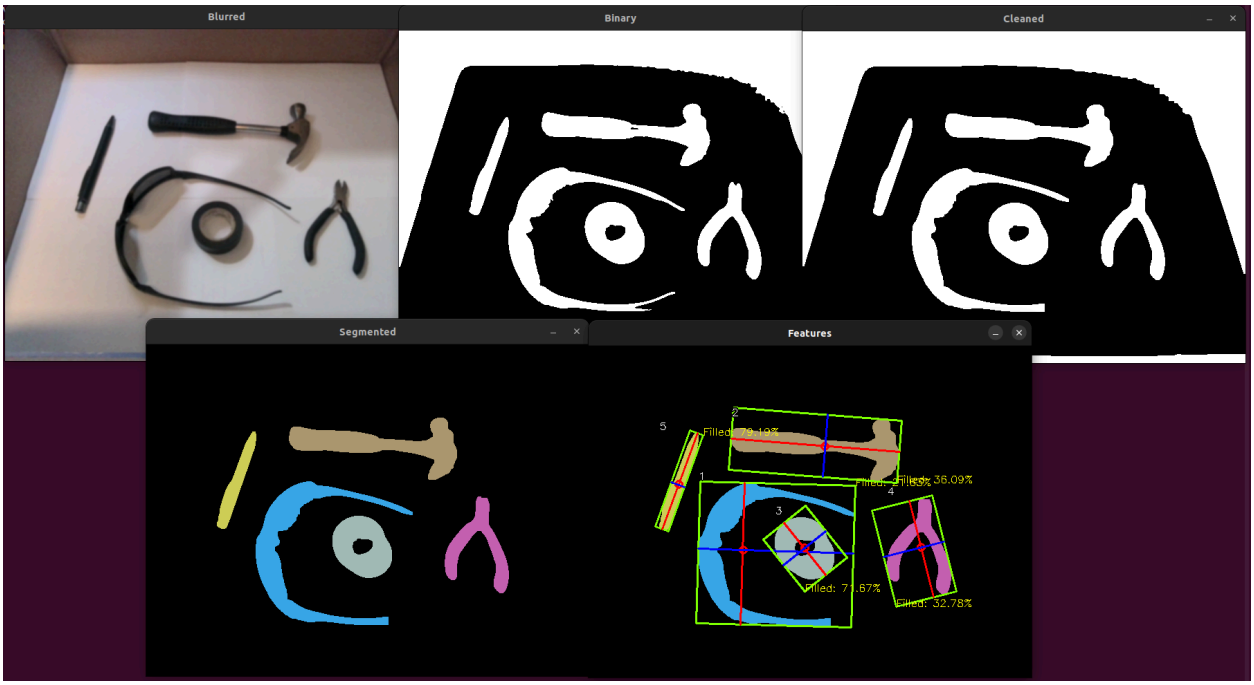
CS 5330 Project 3 Report

Introduction

The purpose of this project was to become familiar with the process of simple object matching / recognition, as well as the necessary pre-processing steps needed to acheive this. This project is primarily concerned with manual object feature embedding, although a CNN-based embedding method (ResNet18) is also compared. The project demonstrates how to threshold an image, perform morphological filtering to clean up the binary image, segment an image, compute the smallest bounding box around an object by calculating the axis of least moment. Lastly, this project demonstrates how rotation, translation, and scale invariant features can be created using this simple information, and which can perform quite well on identifying the same objects under different positions, orientations, and scales.

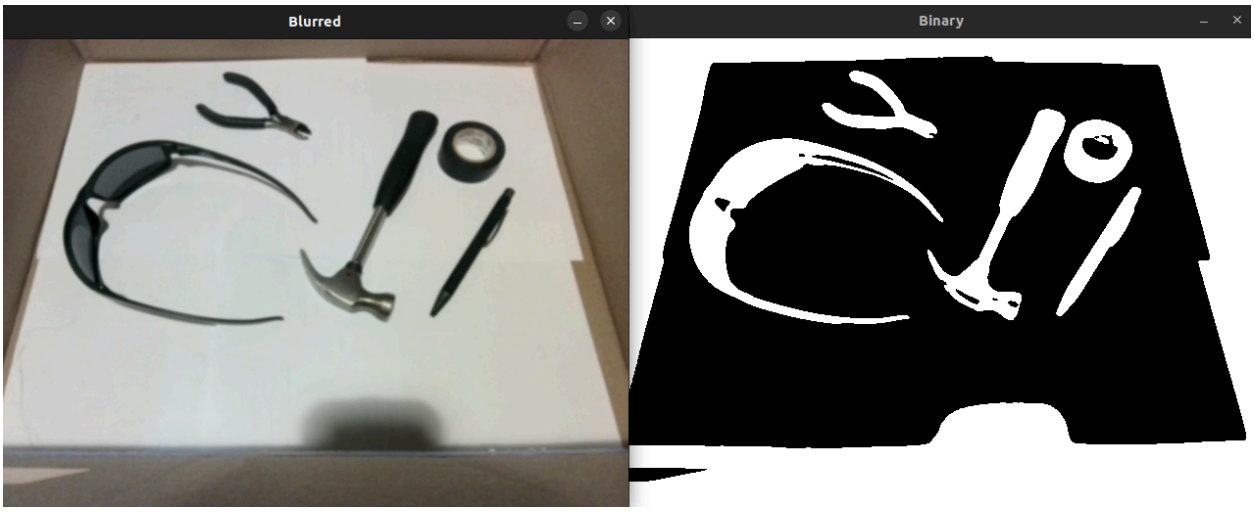


Required Images

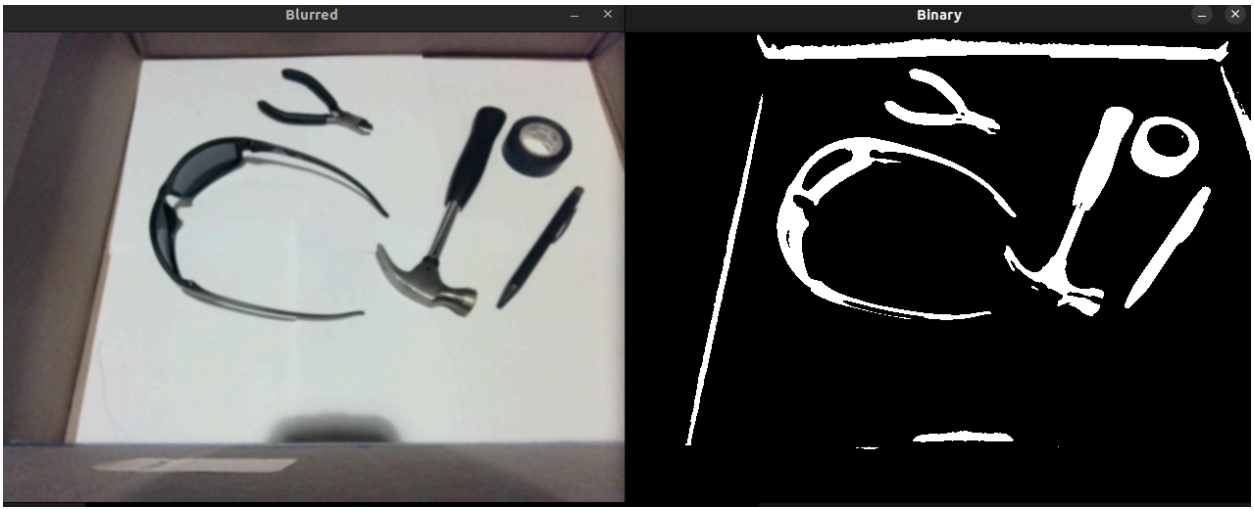
1. Thresholding

I decided to write my own thresholding code. I originally looked at Sauvola thresholding, which is local thresholding normally used for text segmentation, but it had this effect of making the interior of regions hollow, so I switched to using a simple global pixel mean threshold. I also looked at chrominance thresholding (splitting the image into L-a-b channels and removing the Luminance component), converting to greyscale, and then taking a global mean threshold, but this gave a very poor and noisy result.

Global Mean Thresholding:



Sauvola Thresholding:



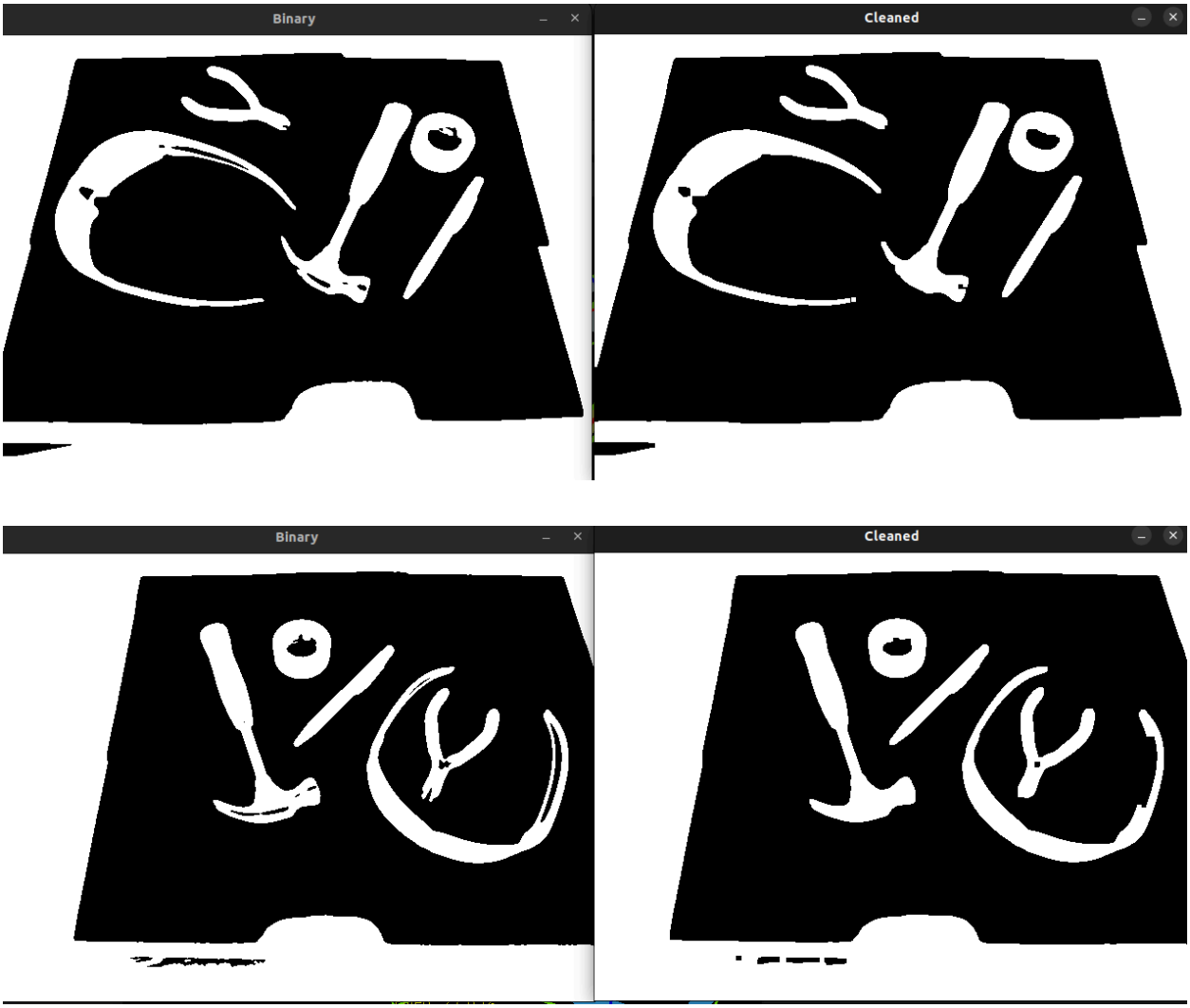
Chrominance Thresholing:

2. Cleaned

To clean the image I used the cv::dilate and cv::erode functions. I performed one dilate, two erodes, and one dilate to effectively both bridge regions and close holes

```
    int num_passes = 2;
    for (int i = 0; i < num_passes; i++) {
        cv::dilate(temp, temp, kernel);
    }
    for (int i = 0; i < num_passes; i++) {
```

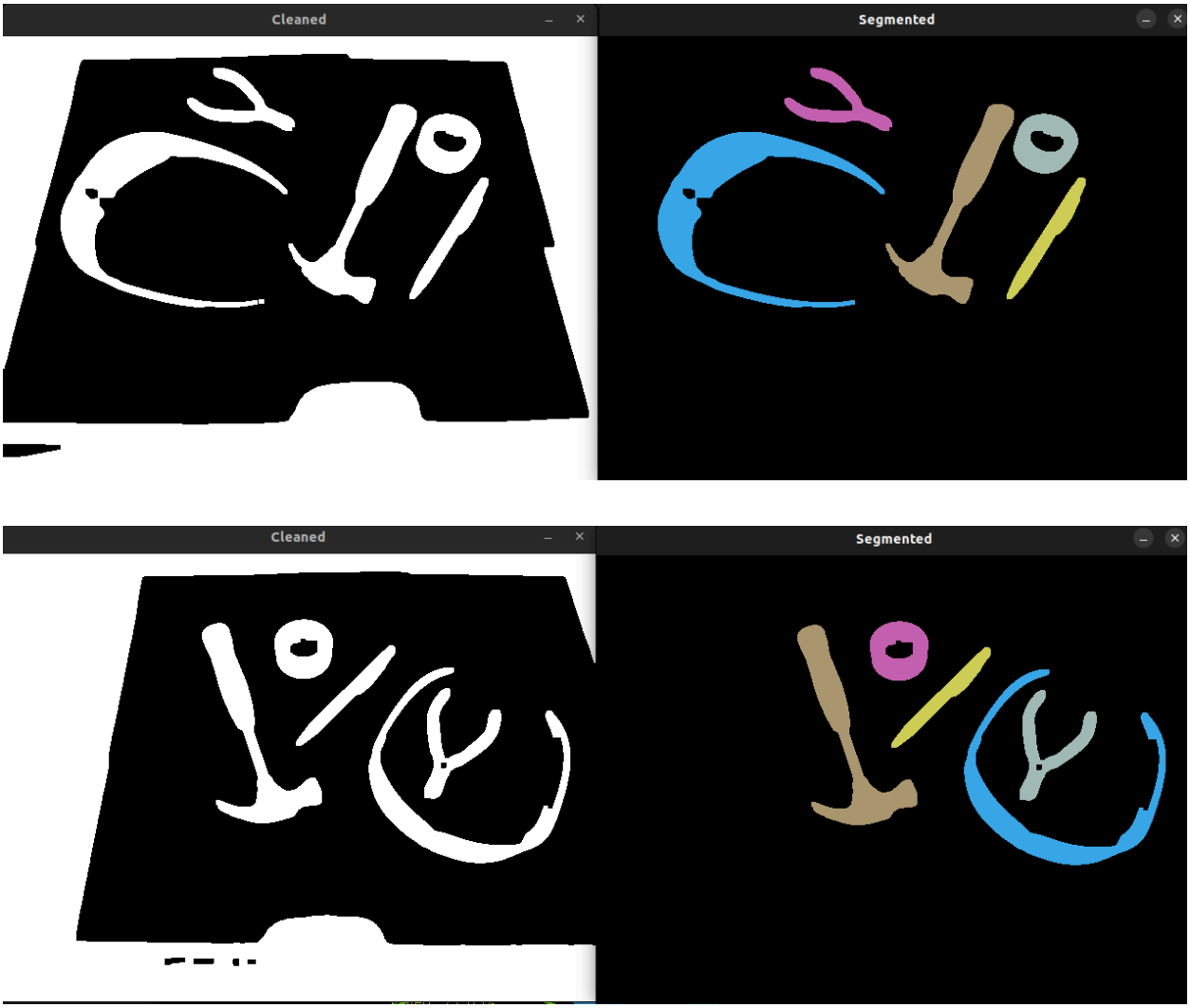
```
        cv::erode(temp, temp, kernel);
    }
    for (int i = 0; i < num_passes; i++) {
        cv::erode(temp, temp, kernel);
    }
    for (int i = 0; i < num_passes; i++) {
        cv::dilate(temp, temp, kernel);
    }
```



3. Segmented

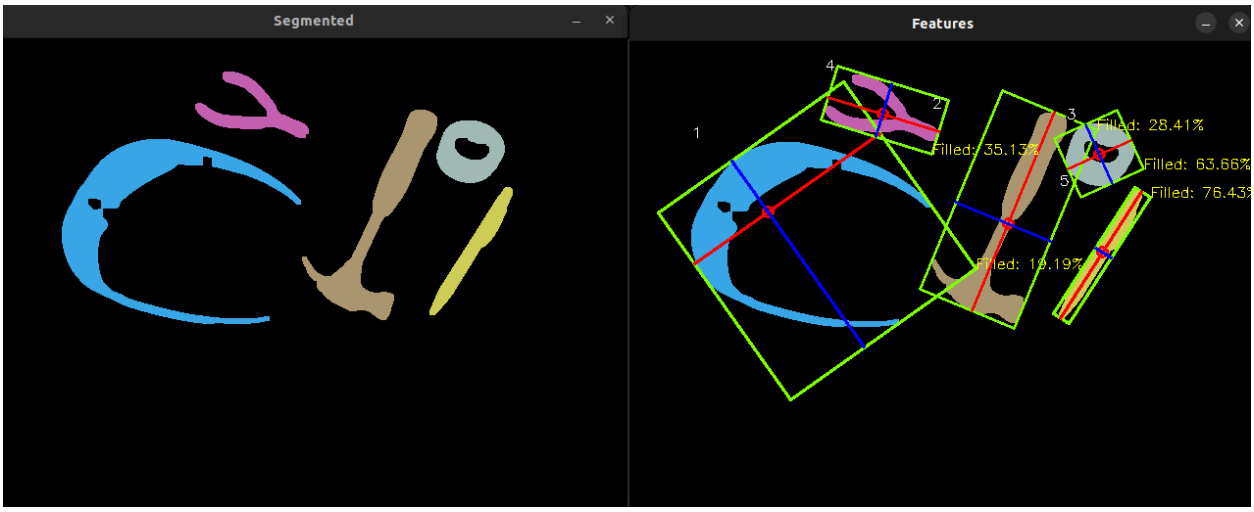
I used cv::connectedComponentsWithStats to segment the thresholded image and return a labels vector and a stats vector, which I use extensively in the later tasks.

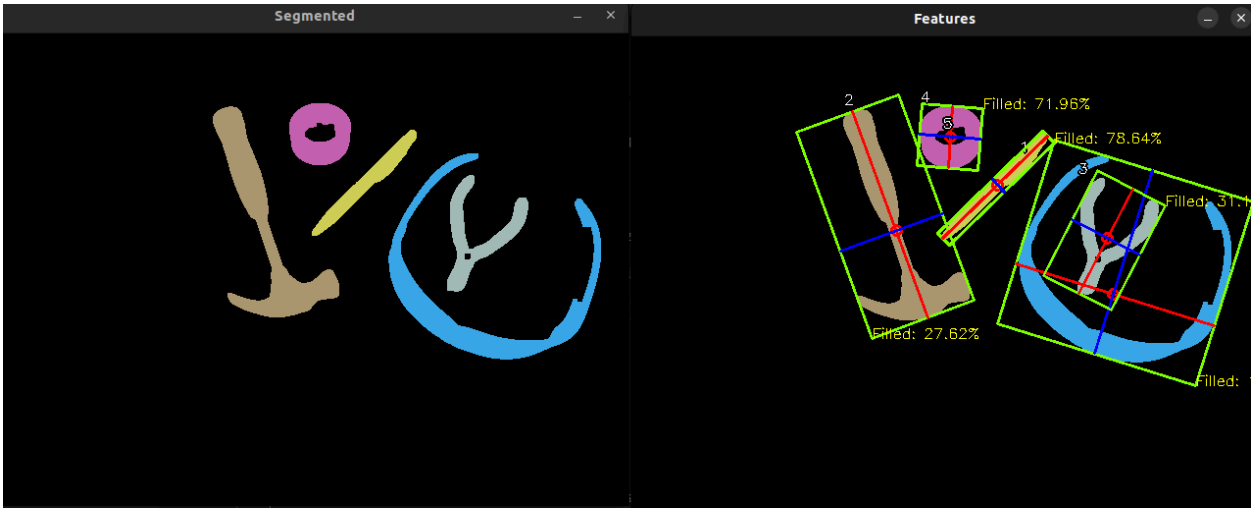
Then I manually remove segments that are too small, I remove regions with a pixel that touches the edge of the image, and I make sure color flicking does not occur by keeping a history of previous region centroids.



4. Features

I implemented my own computation of the axis of least moment. I compute default (m00, m10, m01, m11, m20, m02) and central moments (mu11, mu02, mu20), angle of least central moment, build eigenvectors, and compute the min/max pixel values along those perpendicular eigenvectors. I also use these values to compute corners for the green bounding box.





The features for these objects after one training run are shown below as they appear in the object database csv. I used the aspect ratio of the bounding box and the percent of the bounding box that was filled by the object as my two features.

Object name	Aspect ratio (axis of least moment /axis of most moment)	Percent filled (colored pixels / total bounded pixels)
glasses	0.742015	0.148518
hammer	0.434563	0.304313
tape	0.986144	0.594387
pliers	0.934537	0.252406
pen	0.12476	0.791891

5. Training

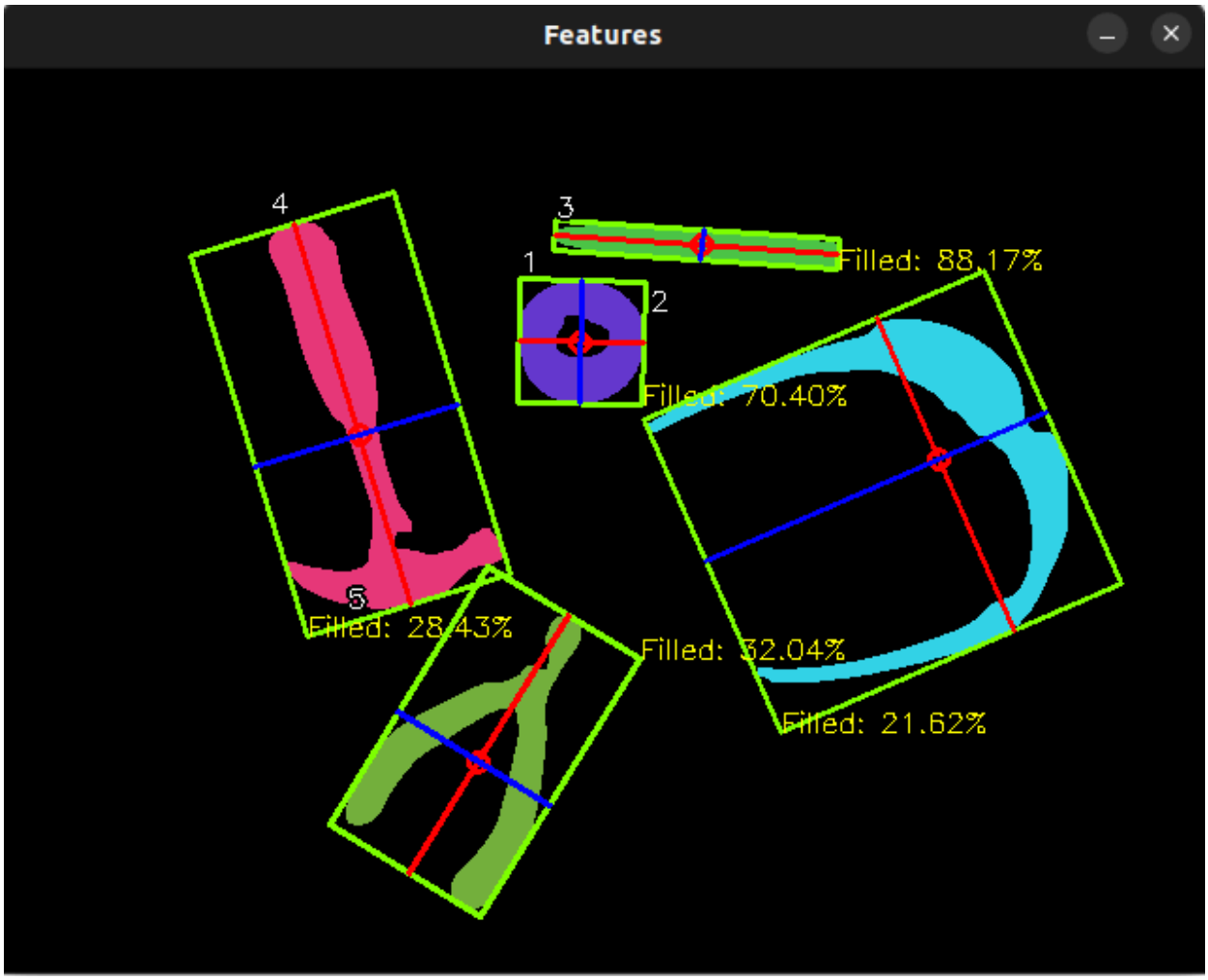
See the “Extensions” section for more detail on how training works. I chose to mix the capability of training / labeling each object with the same pipeline that is used to classify an object. If after computing the distances and sorting the closest object matches in the database, if the distance of the top match is greater than a threshold, it prompts the user to enter the object name and either adds it to the database (or overwrites an existing object if there is already one with the same name - can think of this as ‘refining’ the embedding of an object if a prior embedding was messed up).

6. Classify New Images

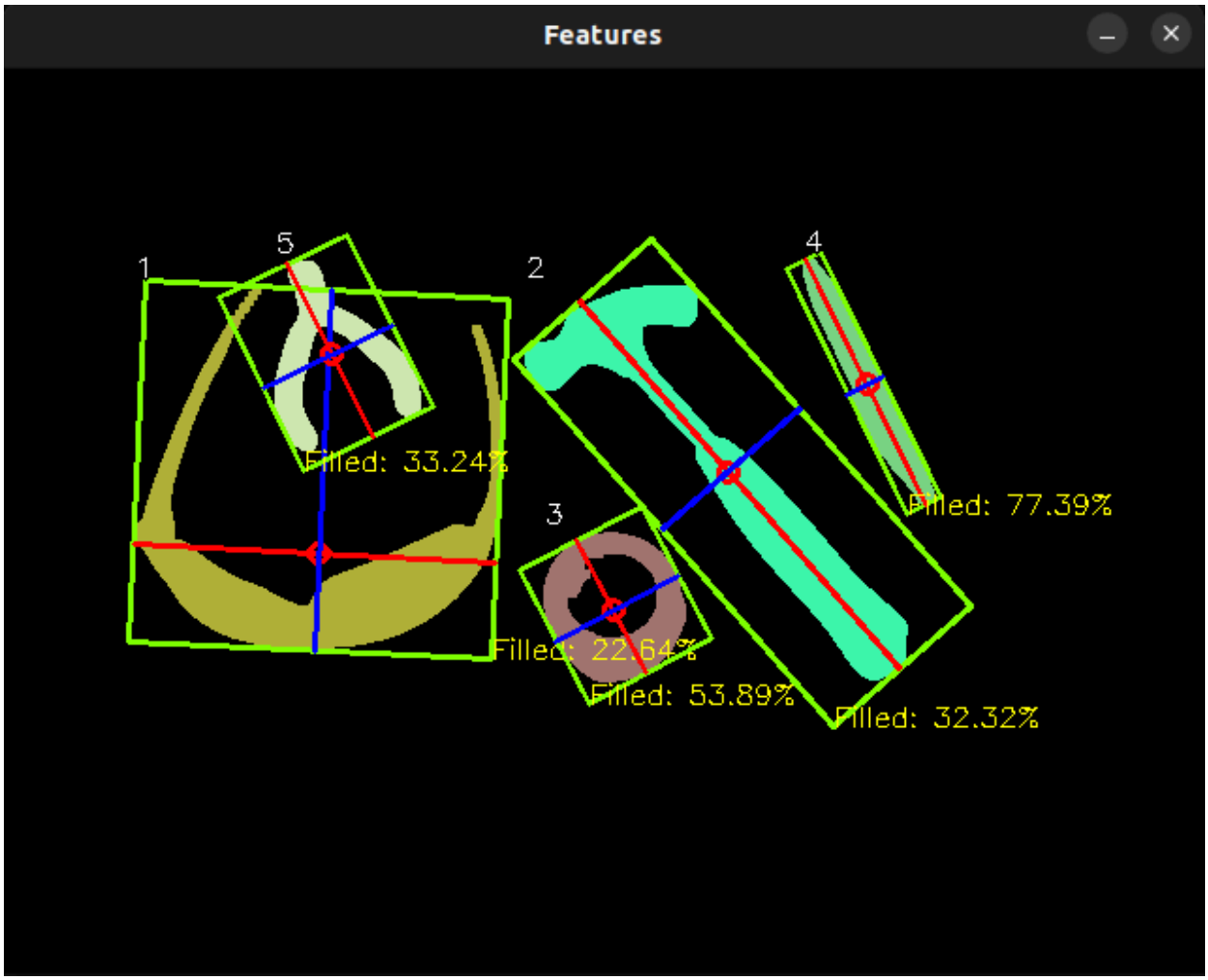
Like explained above, the top 5 matches are shown in the command line for each region in the segmentation image. An example command line output can be seen in the ‘Evaluation’ section, or in the linked demo video.

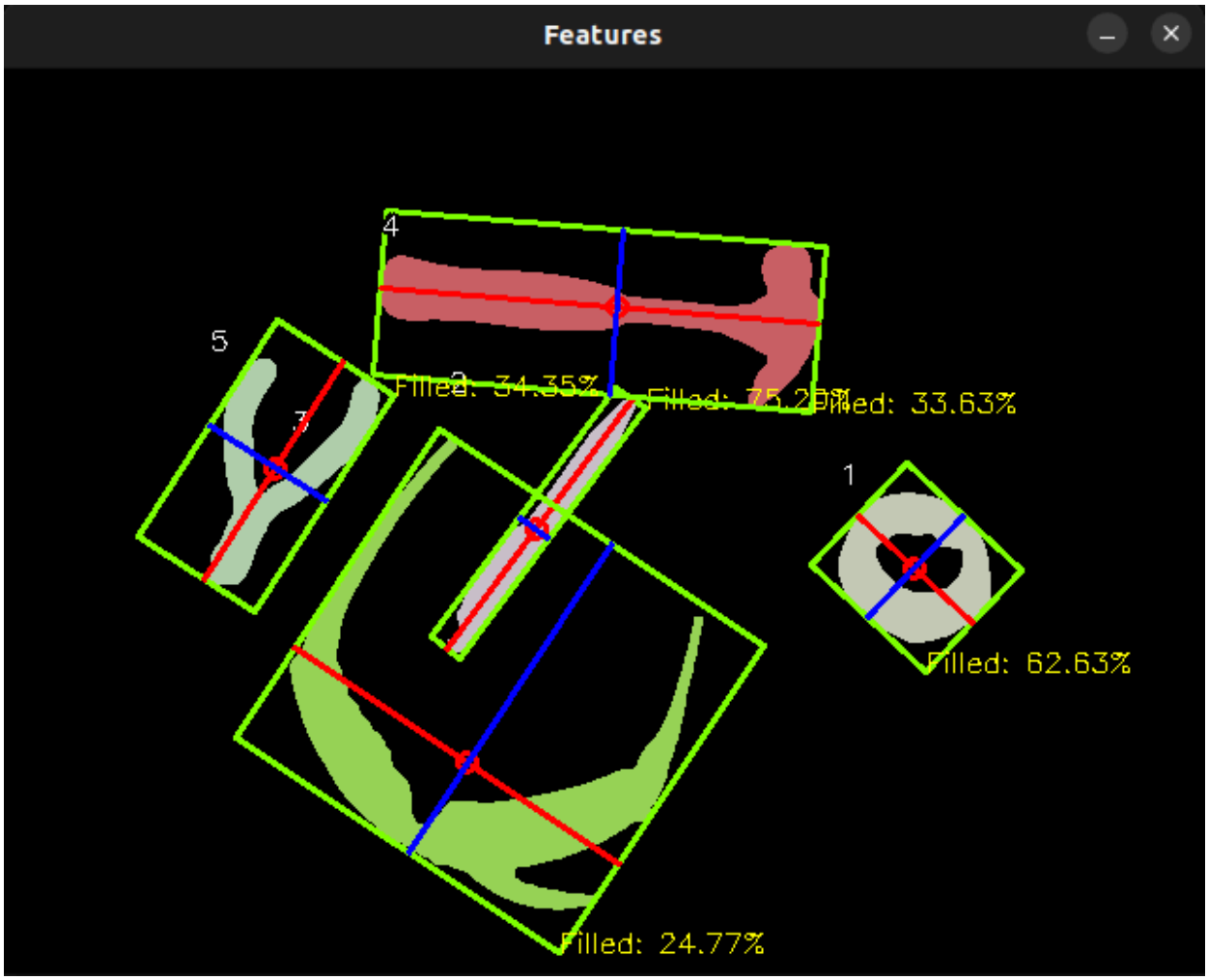
7. Evaluation Confusion Matrix

Below are screenshots for the remaining three configurations I used to evaluate the classification performance and return a confusion matrix.



This above improperly classified the hammer as pliers. I presume that due to the angle of the hammer relative to the camera in this configuration, it made the aspect ratio look smaller than it was and made the hammer AR appear more squat like the pliers. This issue could be resolved with better, orientation invariant features.


Features
1
5
2
4
3
Filled: 33.24%
Filled: 77.39%
Filled: 22.64%
Filled: 53.89%
Filled: 32.32%



Confusion Matrix

Confusion Matrix

Actual / Predicted	glasses	hammer	tape	pliers	pen
glasses	4	0	0	0	0
hammer	0	3	0	1	0
tape	0	0	4	0	0
pliers	0	0	0	4	0
pen	0	0	0	0	4

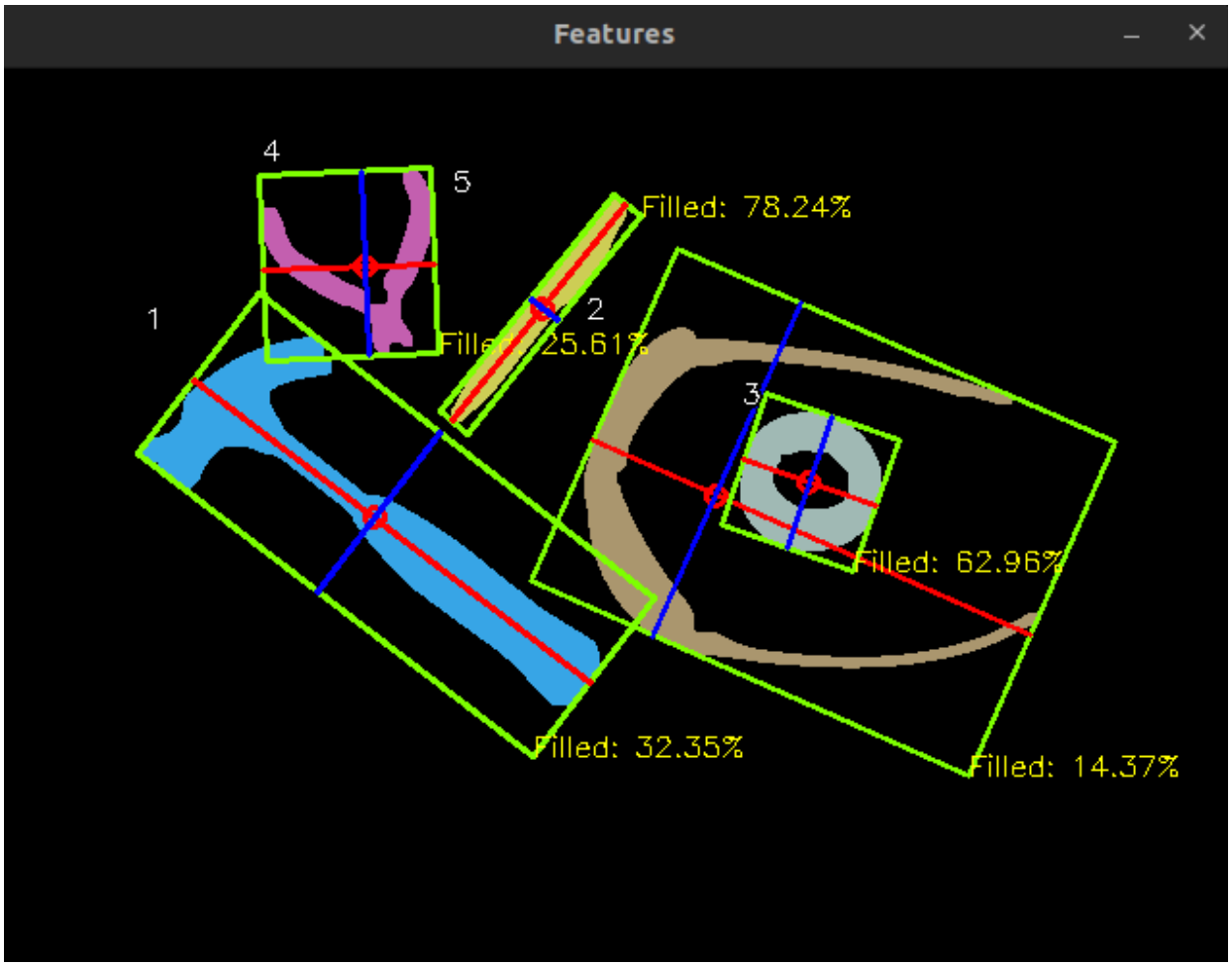
8. Demo video: https://youtu.be/Cbly1P0H3HM

This video walks through the training and classification process within a live video feed.

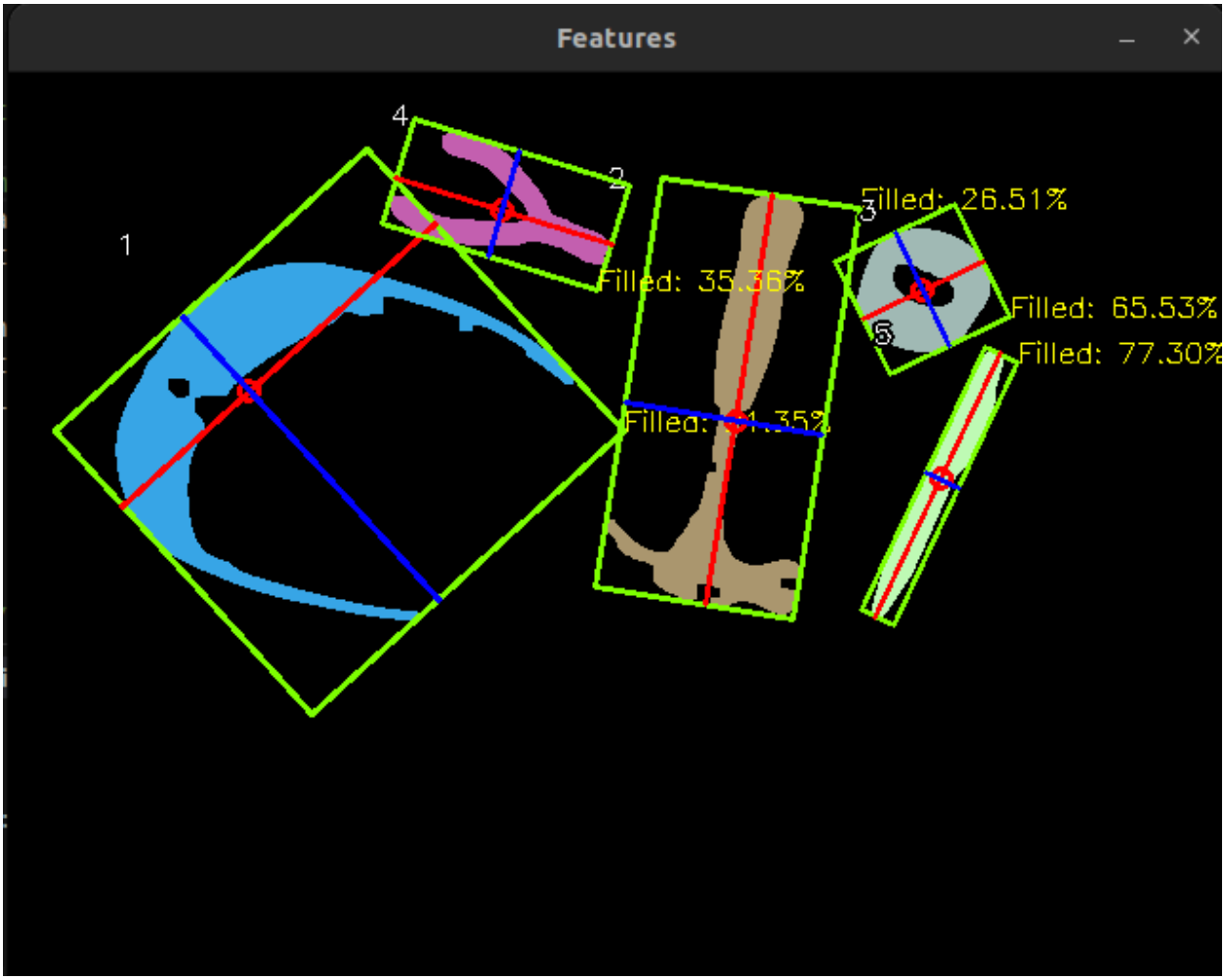
9. One-shot Classification + Comparison

For the one-shot classification, I did a comparison between my handpicked embedding and the ResNet18 embedding. The method of training the ResNet18 system was identical to the previous implementation with classical embeddings (If a object's top match had a distance below a threshold, it prompts the user to enter the object name, and a new row is added to the database). I used scaled euclidean distance as my distance metric like before as well.

I used the arrangements of the same 5 objects below to train the embeddings for each method first.



Then I rearranged the objects into a new configuration to ensure a robust test



After computing the bounding box, I extracted the region of the image inside each bounding box, rotated it such that the primary axis was horizontal, and scaled the image to 224×224.

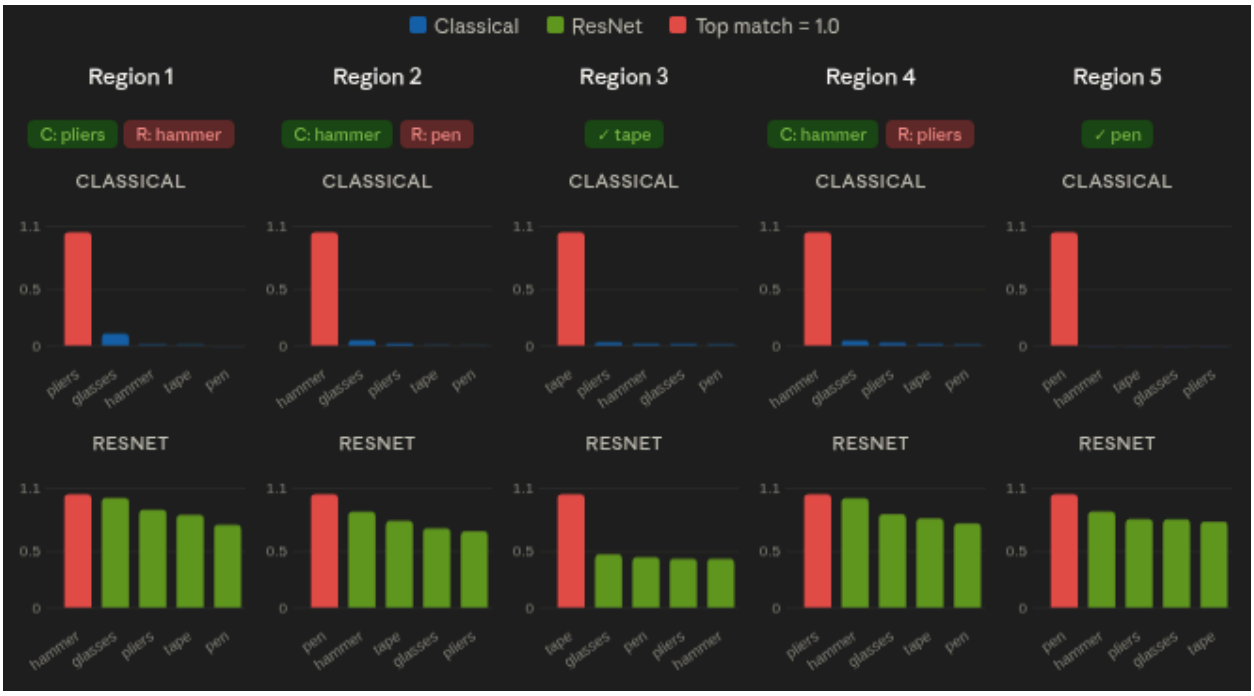
```
Classical:
Top 5 matches for region 1:
  pliers (distance: 0.0358435)
  glasses (distance: 0.321932)
  hammer (distance: 2.23631)
  tape (distance: 2.6183)
  pen (distance: 11.6613)
Recognized object: pliers (distance: 0.0358435)
```

```
Top 5 matches for region 2:
  hammer (distance: 0.0470379)
  glasses (distance: 0.89388)
  pliers (distance: 1.97523)
  tape (distance: 4.36035)
  pen (distance: 6.09625)
Recognized object: hammer (distance: 0.0470379)
Top 5 matches for region 3:
  tape (distance: 0.0954912)
  pliers (distance: 2.81085)
  hammer (distance: 4.40538)
  glasses (distance: 4.77535)
  pen (distance: 6.42663)
Recognized object: tape (distance: 0.0954912)
Top 5 matches for region 4:
  hammer (distance: 0.0673095)
  glasses (distance: 1.36222)
  pliers (distance: 2.0961)
  tape (distance: 3.40874)
  pen (distance: 4.63437)
Recognized object: hammer (distance: 0.0673095)
Top 5 matches for region 5:
  pen (distance: 0.0039914)
  hammer (distance: 4.84407)
  tape (distance: 7.64879)
  glasses (distance: 10.5782)
  pliers (distance: 11.0853)
Recognized object: pen (distance: 0.0039914)

ResNet:
Top 5 matches for region 1:
  hammer (distance: 1566.76)
  glasses (distance: 1618.7)
  pliers (distance: 1813.62)
  tape (distance: 1912.46)
  pen (distance: 2142)
```

```
Recognized object: hammer (distance: 1566.76)
Top 5 matches for region 2:
  pen (distance: 1348.72)
  hammer (distance: 1594.91)
  tape (distance: 1758.58)
  glasses (distance: 1923.96)
  pliers (distance: 2000.94)
Recognized object: pen (distance: 1348.72)
Top 5 matches for region 3:
  tape (distance: 625.151)
  glasses (distance: 1324.51)
  pen (distance: 1397.02)
  pliers (distance: 1444.04)
  hammer (distance: 1446.07)
Recognized object: tape (distance: 625.151)
Top 5 matches for region 4:
  pliers (distance: 1035.61)
  hammer (distance: 1073.88)
  glasses (distance: 1256.18)
  tape (distance: 1314.32)
  pen (distance: 1393.2)
Recognized object: pliers (distance: 1035.61)
Top 5 matches for region 5:
  pen (distance: 2893.58)
  hammer (distance: 3411.4)
  pliers (distance: 3704.44)
  glasses (distance: 3710.71)
  tape (distance: 3811.57)
Recognized object: pen (distance: 2893.58)
```

I used genAI to create a visualization of the results, making sure to normalize the distances for a fair comparasion (top match was given value of 1, all other matches were valued relative to the top match).



From these results we see that the classical embeddings are much more stable (the top prediction was more separated from the other prediction) and more accurate (predicted all 5 correctly, which ResNet only predicted two correctly). The classical embeddings I chose are simple, but for the objects I chose they are quite good at distinguishing the right objects and the small training size was not a problem. Even despite only having one datapoit, due to the nature of the features, the classical approach is able to have **orientation-agnostic** embeddings. ResNet however likely failed to provide good results because I only trained it on one of each object, and thus the embeddings it came up with were **solely specific for the orientation** the objects were in in the training configuration.

Extensions

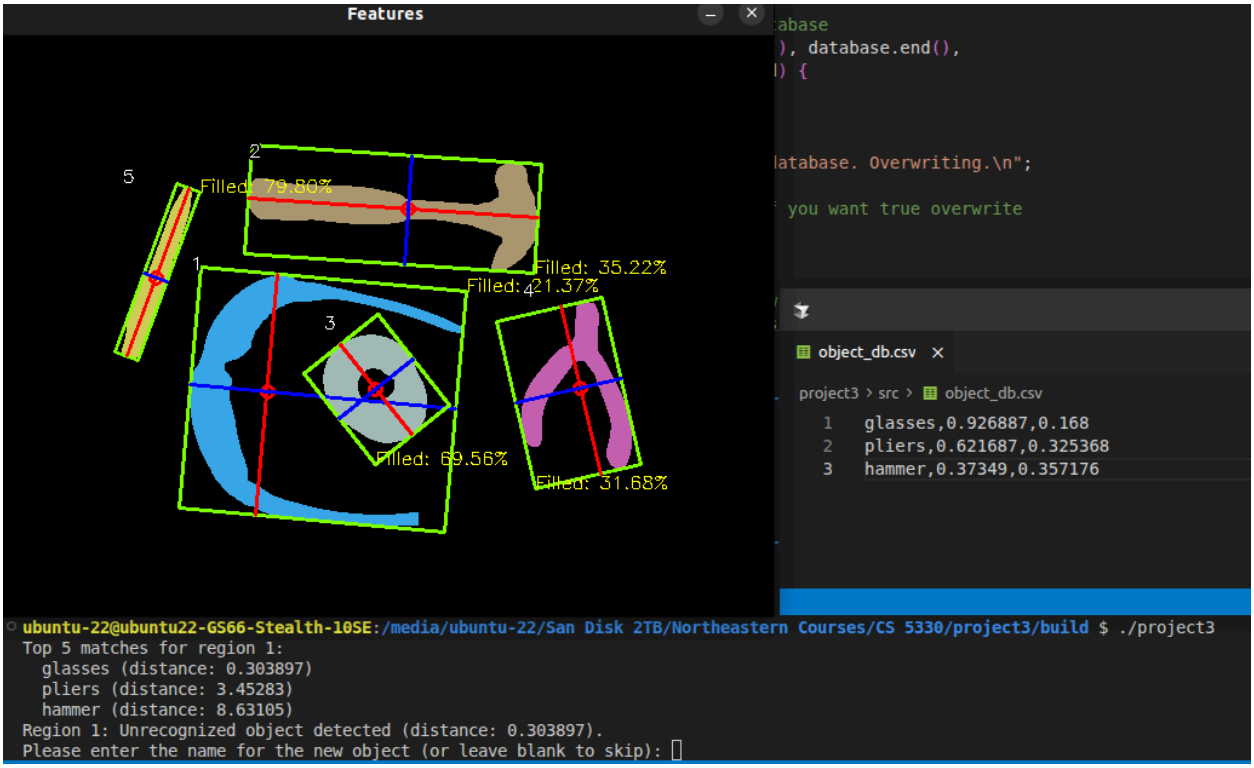
For my first extension, I wrote an extra stage from scratch (wrote total two from scratch). The two stages I wrote are described here:

> Segmentation: Converted the image to greyscale, blurred image, and computed global threshold by taking the mean pixel intensity as the threshold. Also tried converting the image to chrominance, with mixed results

Bounding box computation: computed default and central moments, computed alpha (angle of least central moment), computed eigenvectors, and min/max projection along eigenvector axis to compute bounding box

I also implemented a better method of adding items to the database. When the user presses ‘n’, the euclidean distance of each of the detected objects is computed and compared to a threshold. If a good match is not determined, the video stream freezes, and the user is prompted to enter the name of the object for each unidentified region in the image. If the name input matches the name of an object already in the database, that entry is replaced with the new entry. After all unknown objects have been named, the video stream resumes.

This image shows how the top match for image 1 has a distance that is higher than the allowed threshold (I set the threshold = 0.1). The user is then prompted to enter the object name, assuming the object is new. In this case, glasses already exists in the database (but the embedding was off from an earlier training, perhaps due to a slight segmentation discrepancy), so the new embeddings replace the old ones.



Acknowledgements

Gen AI was used for conceptual understanding, to help write the code to access data in the database csv file, for building/displaying the graphical confusion matrix, for displaying text, green bounding boxes, and red/blue eigenvectors on the cv window. No help from other students was taken.